

Intro2SE - Design - 6

Project Name

*The student team is required to complete the **Design Document** for the assigned course project, following the attached template.*



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

ntents

1	Member Contribution Assessment	2
2	Conceptual Model	3
3	Architectural Design	4
1.1	Architecture Diagram	4
1.2	Class Diagram	4
1.3	Class Specifications	6
1.3.1	Class C1	6
4	Data Design	7
1.4	Data Diagram	7
1.5	Data Specification	7
5	User Interface and User Experience Design	9
1.6	Screen Diagram	9
1.7	Screen Specifications	9
1.7.1	Screen "A"	9
1.7.2	Screen "B"	10
6	AI Usage Declaration	11

7	Presentation.....	12
8	Reflective Report.....	13

Software Design

Objectives

This document focus on the following topics:

- ✓ Completing the Software Design Document with the following contents:
 - Conceptual Model
 - Architectural Design
 - Data Design
 - User Interface Design
- ✓ Understanding the Software Design Document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1

Member Contribution Assessment

StudentID 1 - Fullname 1 - Contribution (%)

- Task Evidence: Provide a comprehensive list of tasks managed via **Jira / Git** (or a similar project management tool). For each entry, include: Task Details: Task name and a descriptive summary of the work performed. Visual Evidence: Clear screenshots that explicitly show the Task Name, Assignee, Status, Assigned Date, and Completion Date.

StudentID 2 - Fullname 2 - Contribution (%)

- Task Evidence

Each task must be assigned to and completed by only one student.

The sum of all student contributions is less than or equal to 100%.

The project tasks includes report writing, self-training, and implementation tasks.

2

Conceptual Model

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Present a diagram illustrating the semantic entities within the software. This section can be created using an Extended Entity-Relationship (EER) model generated by **MySQL Workbench** or **PowerDesigner**]

3 Architectural Design

1.1 Architecture Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Present a **system decomposition tree diagram**, indicating the components of the system]

[Present an overall system architecture diagram, illustrating the relationships between the **MAIN components** identified in the system decomposition tree]

[If there are any special aspects of the architecture, such as the application of Design Patterns, the use of Client-Server/Tier/MVC architecture, support for a plug-in mechanism, etc., these should be clearly presented to demonstrate this]

1.2 Class Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

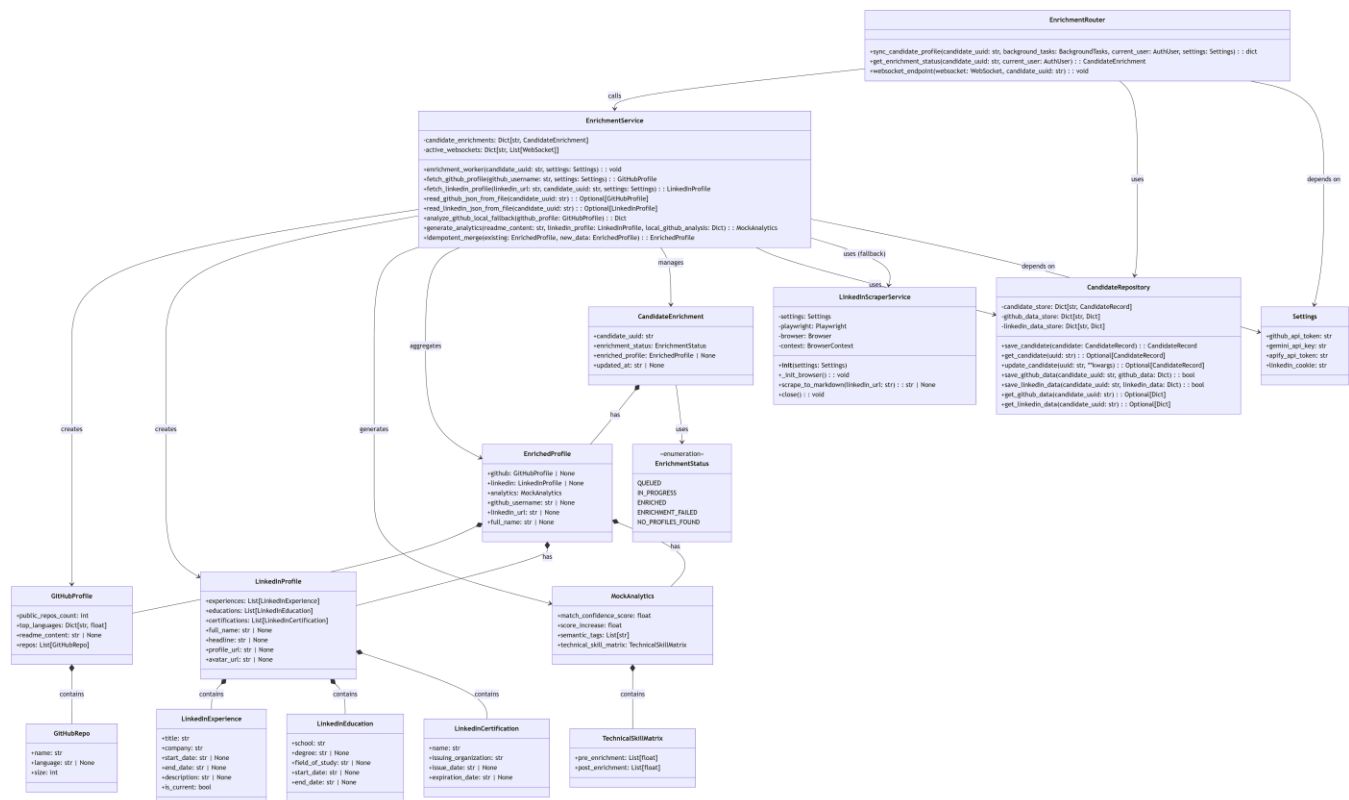
[The class diagram should clearly show:

- The names of the object classes.
- The relationships between the object classes (relationship type, cardinality).

It is not strictly necessary to list the attributes and methods of each object class in the class diagram. However, this information should be included if the diagram is not too large]

1.2.1 Data Crawling Subsystem - Class Diagram (SmartATS Profile Enrichment Module)

The following class diagram documents the actual Data Crawling Subsystem implemented for the Profile Enrichment module of the SmartATS Applicant Tracking System. It shows the classes responsible for fetching, processing, and storing candidate profile data collected from GitHub and LinkedIn.



Link picture: <https://drive.google.com/file/d/1ME0SU92AcvfgJBtO7lsgA1mJ-d62vdK3/view?usp=sharing>

Core service and API classes:

- **EnrichmentService** - orchestrates the enrichment pipeline: fetches GitHub/LinkedIn profiles, reads/writes local JSON caches, runs the local GitHub fallback analysis, generates analytics, and performs the idempotent merge of enriched profiles.
- **EnrichmentRouter** - exposes the FastAPI endpoints: `sync_candidate_profile` (POST `/sync`), `get_enrichment_status`, and the `websocket_endpoint` used for real-time progress updates.
- **CandidateRepository** - persistence gateway that stores candidate records plus raw GitHub/LinkedIn JSON payloads in in-memory dictionaries (`candidate_store`, `github_data_store`, `linkedin_data_store`).
- **LinkedInScraperService** - Playwright-based browser automation service used as a scraping fallback, capable of converting a LinkedIn profile page into Markdown.
- **Settings** - centralised configuration object holding the GitHub API token, Gemini API key, Apify API token, and LinkedIn cookie.

Domain / data model classes:

- **GitHubProfile & GitHubRepo** – represent the public repository count, top languages, concatenated README content, and the list of individual repositories fetched from GitHub.
- **LinkedInProfile, LinkedInExperience, LinkedInEducation, LinkedInCertification** – represent the candidate's professional identity, work history, education history, and certifications scraped from LinkedIn.
- **MockAnalytics & TechnicalSkillMatrix** – hold the derived match-confidence score, score increase, semantic tags, and the pre-/post-enrichment technical skill radar values.
- **EnrichedProfile** – the aggregate root that composes GitHubProfile, LinkedInProfile, and MockAnalytics for a single candidate.
- **CandidateEnrichment & EnrichmentStatus** – track the lifecycle of an enrichment job for a `candidate_uuid` through the states `QUEUED`, `IN_PROGRESS`, `ENRICHED`, `ENRICHMENT_FAILED`, and `NO_PROFILES_FOUND`.

1.3 Class Specifications

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Students should select and present the specifications of a few (9-10) of the most important classes.]

1.3.1 Class C1

[Which class does this class inherit from (if any)]

[Describe the list of attributes (which attributes are inherited – if any, the attribute's visibility: **public**, **protected**, or **private**, etc.)]

Seq	Property	Modifier	Constraint	Description

[List of the main methods/operations]

Seq	Operation	Modifier	Constraint	Description

1.3.2 GitHubScraper Class Specification

Note: the current implementation performs this logic inside the fetch_github_profile function of EnrichmentService rather than a dedicated class. The specification below documents the equivalent functionality as if it were encapsulated in a standalone GitHubScraper class, following object-oriented design principles.

Purpose: Encapsulates GitHub API interaction logic for fetching candidate repository data, README content, and language statistics. Provides caching to minimise API calls and implements fallback strategies for error handling.

Inheritance: None (standalone utility class).

Attributes:

Seq	Property	Modifier	Constraint	Description
1	settings	private	Required	Instance of Settings holding API configuration (github_api_token).
2	cache_dir	private	Default: <code>"/.stored_data"</code>	Directory path for storing cached GitHub data files.
3	cache_ttl_seconds	private	Default: 3600	Time-to-live for cached data, in seconds (1 hour).
4	max_repos	private	Default: 100	Maximum number of repositories fetched per request.
5	timeout_seconds	private	Default: 10.0	HTTP request timeout, in seconds.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	<code>__init__(settings, cache_dir)</code>	public	Required	Initialises the scraper with configuration and cache directory; creates the cache directory if missing.
2	<code>fetch_profile(github_username)</code>	public	Async, Required	Cache-first fetch of the full GitHub profile; falls back to a live API call on cache miss/expiry.

3	<code>_fetch_from_cache(username)</code>	private	Async, Optional	Loads a cached profile from a local JSON file; returns None if missing, corrupted, or expired.
4	<code>_fetch_from_api(username)</code>	private	Async, Required	Performs the live GitHub API call, using the auth token when available; handles rate limits.
5	<code>_parse_repos(repos_data)</code>	private	Sync, Required	Maps the raw GitHub API response into GitHubRepo objects (name, language, size).
6	<code>_calculate_top_languages(repos)</code>	private	Sync, Required	Computes the top 3 languages by code size as percentages.
7	<code>_fetch_readme_content(username, repos)</code>	private	Async, Optional	Fetches README.md content from up to 5 recently updated repositories and concatenates it.
8	<code>_save_to_cache(username, profile)</code>	private	Sync, Optional	Persists the fetched profile to the local JSON cache with a TTL timestamp.
9	<code>_build_headers()</code>	private	Sync, Required	Builds the HTTP headers, adding a Bearer Authorization header when configured.
10	<code>invalidate_cache(username)</code>	public	Sync, Optional	Deletes the cached data for a username to force a fresh fetch.

Exception handling:

- **GitHubAPIException** – raised when the GitHub API returns a non-2xx status code or the rate limit is exceeded.
- **CacheException** – raised when cache file read/write operations fail or the cache is corrupted.
- **NetworkException** – raised on HTTP client network-level errors such as timeouts or connection failures.

Design considerations:

- **Cache-first strategy** – minimises GitHub API quota usage by checking the local cache before calling the API.
- **Async/await pattern** – all I/O (HTTP requests, file I/O) is asynchronous to avoid blocking the event loop.
- **Rate-limit awareness** – respects GitHub's 5000 requests/hour (authenticated) and 60 requests/hour (unauthenticated) limits.
- **Graceful degradation** – returns partial data when some operations fail, e.g. README fetch fails but repository data still succeeds.
- **Thread safety** – cache file operations use file locking to prevent race conditions under concurrent access.

1.3.3 LinkedInScraperService Class Specification

Note: in the current implementation, LinkedIn data is fetched through the `fetch_linkedin_profile` function (Apify actor integration) rather than through this class; browser automation via Playwright is kept as a fallback path. The specification below documents both responsibilities as a single encapsulated class, following object-oriented design principles.

Purpose: Provides LinkedIn profile acquisition for a candidate, primarily by invoking the Apify actor `GOvL4O4RwFqsdIqXF` (proxy-backed scraping), with a Playwright-driven browser-automation fallback that converts a LinkedIn profile page into Markdown when the actor is unavailable or fails.

Inheritance: None (standalone utility class).

Attributes:

Seq	Property	Modifier	Constraint	Description
1	settings	private	Required	Instance of Settings holding apify_api_token and linkedin_cookie configuration.
2	playwright	private	Required	Active Playwright instance used to drive the headless browser.
3	browser	private	Required	Browser instance (Chromium) launched for scraping sessions.
4	context	private	Required	BrowserContext holding session cookies (e.g. linkedin_cookie) for authenticated scraping.
5	cache_dir	private	Default: "./stored_data"	Directory used to cache scraped/actor LinkedIn JSON payloads per username.

Methods:

Seq	Operation	Modifier	Constraint	Description
1	__init__(settings)	public	Required	Stores configuration; the browser/context are created lazily via _init_browser().
2	_init_browser()	private	Async, Required	Launches Playwright/Chromium and creates an authenticated BrowserContext using linkedin_cookie.
3	fetch_profile(linkedin_url,	public	Async,	Primary entry point:

	candidate_uuid, settings)		Required	normalises the URL, checks the local JSON cache, then calls the Apify actor GOvL4O4RwFqsdlqXF via ApifyClientAsync; maps the actor's nested response into a LinkedInProfile (experiences, educations, certifications).
4	scrape_to_markdown(linkedin_url)	public	Async, Optional	Browser-automation fallback: navigates to the profile page with Playwright and returns the rendered page content as Markdown, or None if navigation/parsing fails.
5	_save_to_cache(username, profile)	private	Sync, Optional	Persists the resulting LinkedInProfile as JSON to the local cache to avoid repeated Apify calls.
6	close()	public	Async, Required	Closes the BrowserContext, Browser, and Playwright instance to release resources.

Exception handling:

- **ApifyActorException** – raised when the Apify actor run fails, returns an empty dataset, or the APIFY_API_TOKEN is missing.
- **PlaywrightNavigationException** – raised when the fallback browser cannot load or parse the LinkedIn profile page.
- **CacheException** – raised when the local JSON cache file cannot be read or written.

Design considerations:

- **Cache-first strategy** – checks `./stored_data/cache_linkedin_{username}.json` before calling Apify, reducing scraping cost and latency.
- **Proxy support** – the Apify actor run uses Apify's managed proxy infrastructure to reduce the risk of LinkedIn IP blocking.
- **Graceful, deterministic fallback** – if the Apify actor call raises an exception, `fetch_linkedin_profile` returns a structured fallback `LinkedInProfile` (fixed sample experiences/education) rather than failing the whole enrichment pipeline.
- **URL normalisation** – LinkedIn URLs are standardised (forcing the `www` subdomain and a trailing slash) before being sent to the actor or used as a cache key.
- **Resource cleanup** – `close()` must always be invoked (e.g. in a `finally` block) to avoid leaking browser processes when the Playwright fallback path is used.

1.3.3 Supporting Domain Model Classes

The classes below complete the Data Crawling Subsystem; each supports `EnrichmentService` and `GitHubScraper` by modelling a specific piece of enriched candidate data.

Class	Description
<code>EnrichmentService</code>	Central orchestrator: fetches GitHub/LinkedIn data, generates analytics, applies the idempotent merge, and updates enrichment status.
<code>GitHubProfile / GitHubRepo</code>	GitHub profile aggregate (<code>public_repos_count</code> , <code>top_languages</code> , <code>readme_content</code> , <code>repos</code>) and per-repository metadata (<code>name</code> , <code>language</code> , <code>size</code>).
<code>LinkedInProfile</code>	LinkedIn profile aggregate holding experiences, educations, certifications, <code>full_name</code> , <code>headline</code> , <code>profile_url</code> and <code>avatar_url</code> .
<code>LinkedInExperience</code> /	Individual work, education, and certification records scraped

LinkedInEducation LinkedInCertification	/	from LinkedIn.
MockAnalytics TechnicalSkillMatrix	/	Derived analytics: match_confidence_score, score_increase, semantic_tags, and the pre-/post-enrichment skill matrix.
EnrichedProfile		Aggregate root combining GitHubProfile, LinkedInProfile and MockAnalytics for one candidate.
CandidateEnrichment EnrichmentStatus	/	Tracks the lifecycle of an enrichment job (QUEUED → IN_PROGRESS → ENRICHED / ENRICHMENT_FAILED / NO_PROFILES_FOUND).
CandidateRepository		In-memory persistence gateway for candidate records and raw GitHub/LinkedIn JSON payloads.
LinkedInScraperService		Playwright-driven browser automation used as a fallback scraper, converting a profile page to Markdown.

4

Data Design

1.4 Data Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

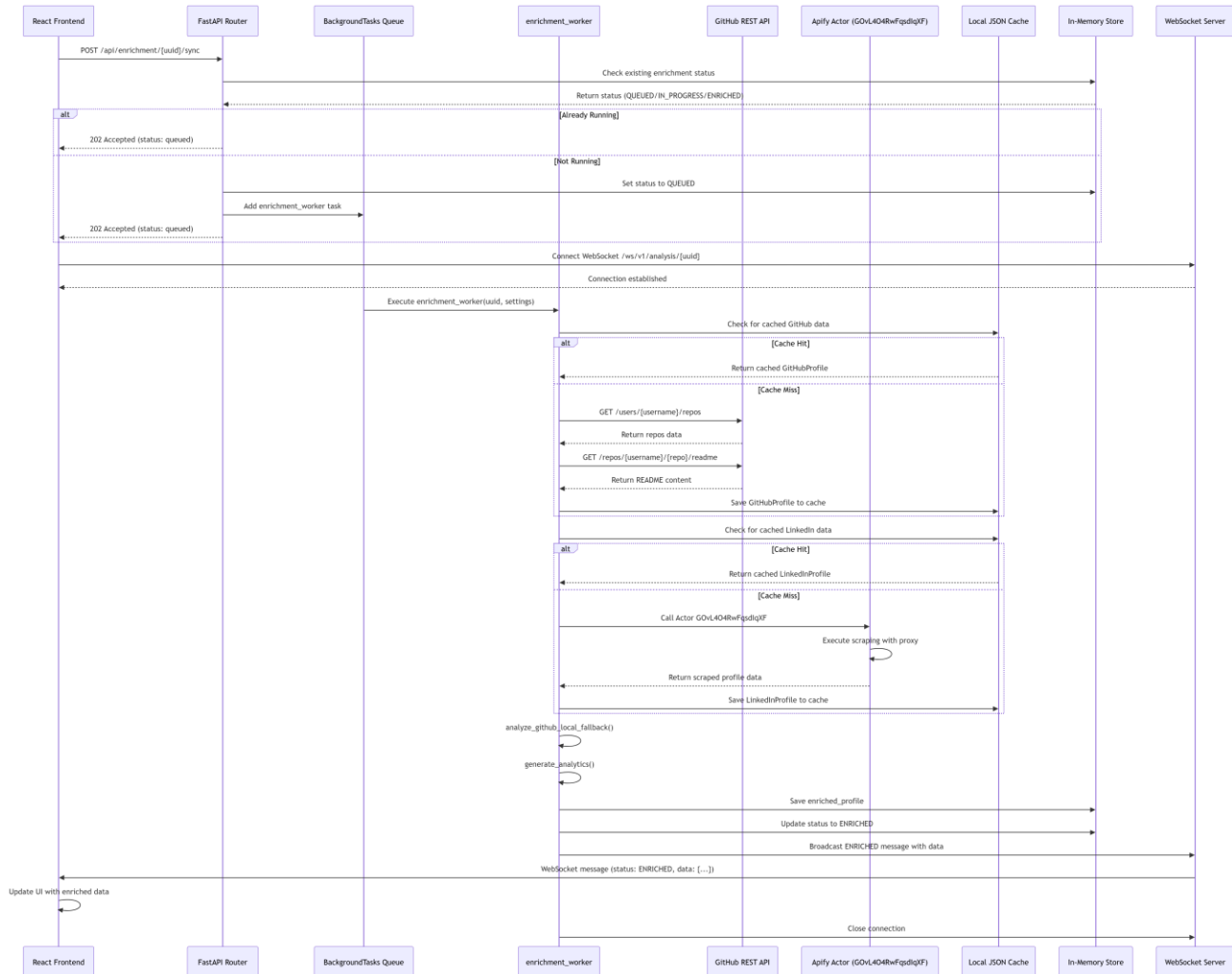
Edited by:

Reviewed by:

[Draw the data diagram of the system, identifying the data components that need to be stored and illustrating the relationships between them]

1.4.1 Asynchronous Crawling Architecture – Enrichment Data Flow

The Profile Enrichment Module implements a hybrid asynchronous architecture that combines FastAPI BackgroundTasks for server-side processing with a WebSocket channel for real-time client updates. This design keeps API responses non-blocking while still giving the recruiter live progress feedback while a candidate's profile is being enriched.



Link

picture:

https://drive.google.com/file/d/1B7mpe0MZi8azuxWbfz4Z1cGSXTMJGQVW/view?usp=drive_link

Enrichment sequence (client → API → background worker → external sources → WebSocket):

- 1. The React frontend sends POST /api/enrichment/{uuid}/sync; the API checks the existing enrichment status in the in-memory store.
- 2. If an enrichment is already QUEUED or IN_PROGRESS, the API immediately returns 202 Accepted with status "queued"; otherwise it sets the status to QUEUED and schedules the enrichment_worker background task, then returns 202 Accepted.

- 3. The frontend opens a WebSocket connection to `/ws/v1/analysis/{uuid}` to receive the final result.
- 4. The background worker checks the local JSON cache for GitHub data; on a cache miss it calls the GitHub REST API for repositories and README content, then writes the result back to the cache.
- 5. The worker checks the local JSON cache for LinkedIn data; on a cache miss it invokes the Apify actor (ID `GOvL4O4RwFqsdIqXF`) through a proxy, then caches the scraped profile.
- 6. The worker runs `analyze_github_local_fallback()` and `generate_analytics()`, merges the new `EnrichedProfile` with any existing one via `idempotent_merge()`, and saves the final profile with status `ENRICHED`.
- 7. The worker broadcasts the `ENRICHED` message (or `ENRICHMENT_FAILED` with an error) to every WebSocket client registered for that candidate, and the frontend updates its UI accordingly.

Key backend components:

- **FastAPI BackgroundTasks queue** – `sync_candidate_profile()` deduplicates in-flight jobs by checking the existing status before queuing `enrichment_worker`, and always returns HTTP 202 immediately.
- **`enrichment_worker()`** – orchestrates the full pipeline: cache-first reads, automatic live-fetch fallback for new candidates, conditional persistence (only when data was freshly fetched from an API), analytics generation, idempotent merge, and WebSocket notification; failures are caught and reported as `ENRICHMENT_FAILED`.
- **Apify actor integration (`fetch_linkedin_profile`)** – checks a local JSON cache first, then calls Apify actor `GOvL4O4RwFqsdIqXF` via `ApifyClientAsync` with proxy support; on any failure it returns a deterministic structured fallback LinkedIn profile so the pipeline never breaks.
- **WebSocket endpoint (`websocket_endpoint`)** – sends the cached result immediately if enrichment already finished, otherwise registers the socket in `active_websockets` for a later broadcast via `notify_websocket_clients()`, and cleans up the registration on disconnect.

Performance optimisations:

- Cache-first strategy reduces external API calls by checking the local JSON cache before any network request.

- GitHub and LinkedIn fetches are logically independent and can be parallelised (currently run sequentially for simplicity).
- HTTPX AsyncClient connection pooling is reused across multiple GitHub API calls.
- Apify's proxy infrastructure rotates IPs to reduce LinkedIn scraping blocks.
- The WebSocket channel only pushes the final result, not intermediate progress, to keep payloads small.

Error handling strategy:

Error Type	Detection	Recovery Mechanism	User Impact
GitHub API rate limit	HTTP response 403	Use cached data or return a partial profile	Delayed enrichment, partial data
Apify actor failure	Empty dataset or exception	Return a structured fallback mock profile	Full profile with mock LinkedIn data
Network timeout	HTTPX timeout exception	Retry with exponential backoff	Slight delay, eventual success
Cache corruption	JSON parse exception	Delete the cache file and refetch	One-time delay, transparent to the user
WebSocket disconnect	onclose event	Client-side reconnection with HTTP status polling fallback	Brief interruption, automatic recovery

1.5 Data Specification

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[If using a Database Management System (DBMS), describe the data tables, the information for each data column including attribute name, data type and value constraints, key constraints..., and explain the attribute]

[If using XML/JSON or a self-defined structured file to store data, the file structure, attribute information, data type, and value constraints need to be specifically described. An example of the content for the information storage file should be included]

1.5.1 Idempotent Merge Strategy for Enriched Candidate Profiles

Repeated enrichment runs for the same candidate (manual re-sync or automatic re-enrichment) must converge on a consistent state without duplicating or losing data. The `idempotent_merge(existing, new_data)` function guarantees this by combining an existing `EnrichedProfile` with newly fetched data field by field, using a different merge strategy per field type.

Merge rules by field:

Field	Merge Strategy	Rationale
github	Prefer new_data, fallback to existing	GitHub data is time-sensitive; the newest fetch is the most accurate.
linkedin.full_name / headline / profile_url / avatar_url	Prefer new_data, fallback to existing	These attributes change as the candidate updates their LinkedIn profile over time.
linkedin.experiences	De-duplicate by company, keep new_data's entry	Company name is treated as a unique key; the newer entry reflects the current role.
linkedin.educations / certifications	Prefer new_data, fallback to existing	Generally stable, but new data may add records.

analytics.semantic_tags	Union of existing and new tags, capped at 8	Tags should accumulate rather than be overwritten, so no discovered skill is lost.
analytics.technical_skill_matrix (pre/post)	Maximum of existing and new, per skill	Both baseline and enriched scores should reflect the best evidence seen so far.
analytics.match_confidence_score	Average of existing and new	Balances confidence across multiple enrichment runs.
analytics.score_increase	Maximum of existing and new	Should reflect the best improvement observed.
github_username / linkedin_url	Prefer new_data, fallback to existing	May be corrected or updated between runs.
full_name	new_data.full_name → new_data.linkedin.full_name → existing.full_name	Cascading fallback keeps a name available whenever any source has one.

Idempotency guarantees:

- **Safe multiple invocations** – calling enrichment repeatedly for the same candidate_uuid converges on the same final state, without duplicate experiences, tags, or skill entries, and without polluting the file system with redundant JSON files.
- **Commutative merge** – for union-based fields (tags) and max-based fields (skill scores) the merge order does not change the result; for preference-based fields (GitHub/LinkedIn profile data) new_data consistently takes precedence.
- **Associative operations** – sequential merges compose: $\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$, enabling incremental updates without corrupting intermediate state.
- **Idempotent identity** – $\text{merge}(X, X) = X$ for any valid enriched profile X – merging a profile with itself produces no change.

Persistence strategy – current implementation:

Candidate enrichment state is kept in an in-memory dictionary (candidate_enrichments) for sub-millisecond access, backed by a local JSON file cache for GitHub and LinkedIn raw payloads:

```
stored_data/  
├─ github_{candidate_uuid}.json  
├─ linkedin_{candidate_uuid}.json  
├─ cache_github_{username}.json  
└─ cache_linkedin_{username}.json
```

Recommended JSONB schema for a future PostgreSQL migration:

```
CREATE TABLE candidate_enrichments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  candidate_uuid VARCHAR(255) UNIQUE NOT NULL,  
  enrichment_status VARCHAR(50) NOT NULL  
    CHECK (enrichment_status IN ('QUEUED', 'IN_PROGRESS',  
      'ENRICHED', 'ENRICHMENT_FAILED', 'NO_PROFILES_FOUND')),  
  enriched_profile JSONB,  
  github_username VARCHAR(255),  
  linkedin_url TEXT,  
  full_name VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
-- GIN index for fast queries inside the JSONB payload  
CREATE INDEX idx_candidate_enrichments_profile_gin  
  ON candidate_enrichments USING GIN(enriched_profile);  
  
-- The UPSERT ON CONFLICT (candidate_uuid) calls an IMMUTABLE  
-- idempotent_merge_jsonb(existing, new_data) PL/pgSQL function  
-- that mirrors the Python idempotent_merge() rules above.
```

Performance considerations:

- GIN indexes enable fast queries within JSONB data structures.
- Partial updates only touch changed fields, reducing database I/O.
- The PostgreSQL merge function is marked IMMUTABLE to help the query planner.
- Local JSON files act as a fast cache layer ahead of any database query.
- Active enrichments are kept in an in-memory store for sub-millisecond access.

Known limitations and recommended enhancements for production:

- **Current limitations** – the in-memory candidate_enrichments dictionary is not persistent across restarts, file-based persistence lacks transactional guarantees, and there is no support for multiple server instances.
- **Recommended enhancements** – replace the in-memory dictionary with Redis for distributed caching, migrate to PostgreSQL with the JSONB schema above, replace BackgroundTasks with Celery/RQ for reliable task execution, add connection pooling (PgBouncer), and offload reads to replicas.

5 User Interface and User Experience Design

1.6 Screen Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Draw a screen diagram illustrating the relationships and transitions between the screens]

[List the screens]

Seq	Screen	Description

1.7 Screen Specifications

[Students should select and present the specifications of a few (5-6) of the most important screens. For the remaining screens, only the user interface design needs to be drawn]

1.7.1 Screen "A"

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Clearly describe the presentation format and the handling of each event within the screen. Include a design image of each screen]

1.7.2 Screen "B"

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Clearly describe the presentation format and the handling of each event within the screen. Include a design image of each screen]

1.7.3 Screen 4 – Enriched Candidate Profile View

State management:

Variable	Type	Initial	Description
loading	boolean	true	Controls the loading display; true while enrichment is in progress.
data	EnrichedProfile null	null	The complete enriched profile once available.
error	string null	null	Error message when enrichment fails.

syncing	boolean	false	Whether a manual sync is currently running.
resolvedRef	React.useRef<boolean>	false	Prevents duplicate resolution once enrichment has completed or failed.
hasTriggeredSyncRef	React.useRef<boolean>	false	Prevents duplicate sync calls on re-renders.
wsRef	React.useRef<WebSocket null>	null	Holds the active WebSocket connection.
reconnectTimeoutRef	React.useRef<Timeout null>	null	Holds the pending reconnection timer for cleanup.

State transitions:

```

[Initial] loading=true, data=null, error=null
  | (WebSocket connects / status check)
[Loading] loading=true, data=null, error=null
  | (WebSocket: ENRICHED)          | (WebSocket: ENRICHMENT_FAILED)
[Success] loading=false, data=set  [Error] loading=false, error=set
  \_____ (User clicks Retry) _____/
                [Retry] loading=true, data=null, error=null
  
```

Key event handling:

Event	Handler	Logic
Page load / mount	useEffect on candidateUuid	Extracts candidate_uuid from the URL, stores it in WorkspaceContext, and triggers the enrichment sync if not already running.

WebSocket connect	connectWebSocket()	Builds ws://.../ws/v1/analysis/{uuid}, closes any existing socket to avoid duplicates, and registers onopen/onmessage/onerror/onclose handlers.
WebSocket message	ws.onmessage	On status ENRICHED: stores the data, marks resolved, closes the socket. On ENRICHMENT_FAILED: sets the error message, marks resolved, closes the socket.
WebSocket unexpected close	ws.onclose	Ignores manual closes or already-resolved cases; otherwise polls the HTTP status endpoint as a fallback and, if still unresolved, reconnects after a 2.5s backoff.
Accordion header click	toggle("github" "linkedin")	Implements single-open accordion behaviour for the GitHub and LinkedIn cards.
Career timeline header click	setExpanded()	Expands/collapses the career timeline, independent of the accordion cards.
"Show Delta" toggle	setShowBoth()	Switches the radar chart between showing only the post-enrichment score or both pre- and post-enrichment for comparison.
Automatic sync on load	triggerSync() in useEffect	Guards against duplicate calls via refs, calls syncCandidateProfile(), and updates the syncing indicator; failures set the error state.
Component unmount	cleanup in useEffect	Closes the WebSocket gracefully (code 1000), marks it as manually closed to prevent reconnection, and clears any pending reconnection timer.

Performance considerations:

- Exponential backoff (2.5s initial delay) on WebSocket reconnection prevents connection spam.

- Sync/connection flags are kept in refs rather than state to avoid unnecessary re-renders.
- A WeakSet tracks manually closed sockets so they can be garbage-collected, preventing memory leaks.
- Components render conditionally, only once their data is available, to reduce DOM operations.

Accessibility features:

- Semantic HTML with a proper heading hierarchy.
- Accordion cards are keyboard-accessible via Tab and Enter.
- Loading and error states are announced through ARIA labels for screen readers.
- All text meets WCAG AA colour-contrast requirements under the design system's palette.

6

AI Usage Declaration

Based on the provided AI Usage Guideline document, teams must submit their AI usage declaration here. Students are encouraged to leverage AI tools creatively and effectively in this project.

7

Presentation

Teams are required to record presentation videos of their work using the provided template.

Every team member must participate in the presentation, specifically covering the sections their contributed to the project.

Videos must not exceed 30 minutes.

Please upload your videos to Youtube (set to either Unlisted or Public).

Provide the Youtube links the the field below.

8

Reflective Report

Which sections of this template are most helpful for implementation, and which are unnecessary? Provide specific examples to support your reasoning.